Journal of Logic, Language and Information

A Substructural Type Theory for Categorial Grammars

--Manuscript Draft--

Manuscript Number:	
Full Title:	A Substructural Type Theory for Categorial Grammars
Article Type:	Manuscript
Keywords:	Categorial Grammar Type Theory Substructural Logic Dependent Type
Corresponding Author:	Zhaohui Luo Royal Holloway University of London Egham, UNITED KINGDOM
Corresponding Author Secondary Information:	
Corresponding Author's Institution:	Royal Holloway University of London
Corresponding Author's Secondary Institution:	
First Author:	Zhaohui Luo
First Author Secondary Information:	
Order of Authors:	Zhaohui Luo
	Stergios Chatzikyriakidis
Order of Authors Secondary Information:	
Funding Information:	
Abstract:	This paper studies a substructural type theory, which contains ordered types and linear types, and their dependent versions as well. The calculus employs standard ‘ordered terms’ of ordered types, as well as linear terms of linear types, which can be used to represent NL phrases/sentences in syntactical analysis. For easier understanding and clearer presentation, we first present its non-dependent part and give simple examples to illustrate its uses in syntactical analysis, and then introduce the full calculus with dependent types and the associated subtyping mechanism. The substructural type theory naturally extends the Curry-Howard correspondence to Lambek ordered types (and their dependent versions) and its nice meta-theoretic properties are briefly discussed as well.

Powered by Editorial Manager® and ProduXion Manager® from Aries Systems Corporation

Click here to access/download;Manuscript;CG23JoLLI (40).pdf

Click here to view linked References

manuscript No.
(will be inserted by the editor)

A Substructural Type Theory for Categorial Grammars

Zhaohui Luo · Stergios Chatzikyriakidis

Received: date / Accepted: date

Abstract This paper studies a substructural type theory, which contains ordered types and linear types, and their dependent versions as well. The calculus employs standard 'ordered terms' of ordered types, as well as linear terms of linear types, which can be used to represent NL phrases/sentences in syntactical analysis. For easier understanding and clearer presentation, we first present its non-dependent part and give simple examples to illustrate its uses in syntactical analysis, and then introduce the full calculus with dependent types and the associated subtyping mechanism. The substructural type theory naturally extends the Curry-Howard correspondence to Lambek ordered types (and their dependent versions) and its nice meta-theoretic properties are briefly discussed as well.

1 Introduction

It has been proposed by Oehrle [29], among others, that the Curry-Howard correspondence [8,11] between formulas/proofs and types/terms can be usefully exploited in a framework of categorial grammar for syntactical analysis. This has led to the study of Girard's linear types $A \multimap B$ [9] in the context of categorial grammar (see, for example, deGroote's ACG [10] and Muskens' λG [28]) and further development of hybrid categorial grammars that combine Lambek ordered types B/A and $A \backslash B$ [16] with linear types in categorial analysis (see, for example, Kubota and Levine's HTLG [14,15,25]) and related approaches (e.g., Morrill's Displacement Calculus [27,26] and Barker's NL_λ [2, 1]).

In this paper, we study a substructural type theory $\bar{\lambda}_\Pi$, which contains not only ordered types and linear types, but also corresponding substructural dependent function types (Π-types), i.e., dependent ordered types and dependent linear types.[1] In the calculus, we consider not only linear terms, as was

[1] A slightly different version of $\bar{\lambda}_\Pi$ is proposed and studied in [19]. $\bar{\lambda}_\Pi$ does not contain intuitionistic function types (either the dependent or non-dependent versions) – that's what

	Π-type	Non-dependent type	Abstraction	Application
Linear	$\overline{\Pi}x{:}A.B$	$A \multimap B$	$\overline{\lambda}x{:}A.b$	$\overline{app}(f,a)$
Ordered (right)	$\Pi^r x{:}A.B$	B/A	$\lambda^r x{:}A.b$	$app^r(f,a)$
Ordered (left)	$\Pi^l x{:}A.B$	$A \setminus B$	$\lambda^l x{:}A.b$	$app^l(a,f)$

Table 1 Three substructural function types in $\bar{\lambda}_\Pi$: summary of notations.

studied by Oehrle [29] and others, but 'ordered terms' of ordered types [30, 18], for both non-dependent and dependent function types.

Linear types/terms are written with the $^-$ sign and, for ordered types and their terms, we use the superscripts 'r' and 'l' to indicate 'right' and 'left', respectively. These notations are summarised in Table 1 and explained as follows:

- $\overline{\Pi}x{:}A.B$, a linear Π-type, has abstraction objects of the form $\overline{\lambda}x{:}A.b$, with corresponding application objects $\overline{app}(f,a)$. When x does not occur free in B, $\overline{\Pi}x{:}A.B$ becomes a non-dependent linear function type, usually written as $A \multimap B$.
- Ordered Π-types:
  - $\Pi^r x{:}A.B$ has abstraction objects of the form $\lambda^r x{:}A.b$, with corresponding application objects $app^r(f,a)$. When x does not occur free in B, $\Pi^r x{:}A.B$ becomes a non-dependent ordered type, usually written as B/A.
  - $\Pi^l x{:}A.B$ has abstraction objects of the form $\lambda^l x{:}A.b$, with corresponding application objects $app^l(a,f)$. When x does not occur free in B, $\Pi^l x{:}A.B$ becomes a non-dependent ordered type, usually written as $A \setminus B$.

The calculus $\bar{\lambda}_\Pi$ naturally extends the Curry-Howard correspondence to Lambek ordered types. In particular, different from other existing work, our terms for ordered types are standard ones in type theory (with binders such as λ^r and λ^l), rather than more loosely related ones such as λ-strings. It has natural properties as expected for such type systems. For example, a well-typed term determines its types; in other words, type inference works as expected: from a well-typed term a in context Γ, one can infer its type(s) A such that $\Gamma \vdash a : A$.

$\bar{\lambda}_\Pi$ will be studied gradually for easier understanding: in Section 2, we shall first introduce its non-dependent part, called $\bar{\lambda}_\rightarrow$, containing only non-dependent substructural function types (linear types $A \multimap B$ and ordered types B/A and $A \setminus B$). The full calculus $\bar{\lambda}_\Pi$ with dependent substructural function types will be introduced in Section 4 where, besides explaining how type dependency (and the related subtyping mechanism) should be introduced for a substructural type theory, we shall also discuss how substructural dependent types can be used in syntactical analysis.

the $^-$ sign in $\bar{\lambda}_\Pi$ means. If intuitionistic function types are added, the resulting system would be called λ_Π, which we do not discuss in this paper (see [20] for such a system of linear types).

Substructural logical systems have been studied as core calculi for categorial grammars.[2] Several features of interest are discussed when our substructural type theory is employed for categorial analysis. For instance, in Section 3, we consider how its terms can be used as representations of natural language phrases or sentences. This was proposed and considered in [18,7], but in this paper, we make this representational scheme clear by giving a simple mathematical mapping from terms to strings. To illustrate how this works, we shall give simple examples of categorial analysis to explain its uses. Among these examples is Example 3 in Section 3: it shows that, when terms in our substructural calculus are used as representations, a seemingly necessary side condition of contextual non-emptiness in the introduction rules as employed in the traditional Lambek categorial grammar [16] (also see, for example, Section 2.5 of [24]) is not needed anymore .

When introducing dependent types into a substructural calculus, a most challenging decision to make is whether to allow types to depend on substructural variables. In most of the existing research so far (see, for example, [5,13, 31,18]), types are only allowed to depend on intuitionistic variables, but not on substructural variables. In $\bar{\lambda}_{\Pi}$, we do allow dependency over substructural variables but, however, it does not have intuitionistic types (or intuitionistic variables) at all. In fact, if we did incorporate intuitionistic Π-types, we would obtain the calculus λ_{Π}, whose formulation is much more sophisticated (see [20] for such a calculus where we only consider linear types and not ordered types). Put in another way, although it allows type dependency over substructural variables, the absence of intuitionistic types in $\bar{\lambda}_{\Pi}$ has led to a simplicity of its formulation as well as its meta-theoretic study. For applications in categorial grammatical analysis, this seems to have struck a right balance.

The paper is organised as follows. Section 2 studies $\bar{\lambda}_{\rightarrow}$, the subsystem of $\bar{\lambda}_{\Pi}$ with only non-dependent ordered and linear function types, but without dependent types. The formulation follows the standard approach in λ-calculus to use λ-binders for ordered types as well as linear types and corresponding conversion laws. Examples are discussed in Section 3 to illustrate how the substructural types can be used in syntactical analysis and to show, in particular, how terms can be used to represent NL phrases and sentences. Substructural dependent function types are introduced in Section 4, where the full calculus $\bar{\lambda}_{\Pi}$ is presented, together with the associated subtyping mechanism, which is essential in syntactical analysis. Metatheoretic properties of $\bar{\lambda}_{\Pi}$ are briefly discussed in Section 5 where we show that the presented substructural calculi do indeed satisfy the linearity property. In the conclusive section, we discuss some related work as well as future work such as further extensions.

[2] As Moortgat once put it [23]: 'Typelogical grammars are substructural logics, designed for reasoning about the composition of form and meaning in natural language.'

Variables

$$\frac{}{x{:}A \vdash x : A}$$

Ordered function types

$$\frac{\Gamma,\ x{:}A \vdash b : B}{\Gamma \vdash \lambda^r x{:}A.b : B/A} \qquad \frac{\Gamma \vdash f : B/A \quad \Delta \vdash a : A \quad FV(\Gamma) \cap FV(\Delta) = \emptyset}{\Gamma, \Delta \vdash app^r(f, a) : B}$$

$$\frac{\Gamma,\ x{:}A \vdash b : B}{\Gamma \vdash \lambda^l x{:}A.b : A \setminus B} \qquad \frac{\Gamma \vdash f : A \setminus B \quad \Delta \vdash a : A \quad FV(\Gamma) \cap FV(\Delta) = \emptyset}{\Delta, \Gamma \vdash app^l(a, f) : B}$$

Linear function types

$$\frac{\Gamma,\ x{:}A \vdash b : B}{\Gamma \vdash \overline{\lambda} x{:}A.b : A \multimap B} \qquad \frac{\Gamma \vdash f : A \multimap B \quad \Delta \vdash a : A \quad FV(\Gamma) \cap FV(\Delta) = \emptyset}{\Gamma, \Delta \vdash \overline{app}(f, a) : B}$$

The lexicon rule

$$\frac{(c, A) \in \textsc{Lex}}{\langle\rangle \vdash c : A}$$

Fig. 1 Inference rules of $\bar{\lambda}_{\rightarrow}$

2 $\bar{\lambda}_{\rightarrow}$: Substructural (Non-dependent) Function Types

In this section, we describe $\bar{\lambda}_{\rightarrow}$, the subsystem of $\bar{\lambda}_{\Pi}$ with only non-dependent ordered function types and linear function types, but not corresponding dependent types. Because there is no dependent type in $\bar{\lambda}_{\rightarrow}$, the notion of type can be defined independently of objects, as follows.

Definition 1 (types in $\bar{\lambda}_{\rightarrow}$) *Types in $\bar{\lambda}_{\rightarrow}$ are inductively defined as follows:*

1. *The basic categories (such as* S, NP *and* CN*) are types.*
2. *If A and B are types, so are $A \multimap B$, B/A and $A \setminus B$.* □

We assume that the lexicon Lex be a finite set of pairs (c, A), where c is a constant and A is a type. Note that there may be more than one pair for the same constant. In other words, a constant may be assigned more than one type in the calculus. Also, contexts Γ are sequences of the form $x_1{:}A_1, ..., x_n{:}A_n$ $(n \geq 0)$ with $A_i's$ being types and x_i's being distinct variables. When $n = 0$, we write $\langle\rangle$ for the empty context.

In a judgement $\Gamma \vdash a : A$ in a type theory (or a typed λ-calculus), Γ is a context, A is a type and a is called a term (or object). The judgement means that, under context Γ, a is an object of type A.

The inference rules of $\bar{\lambda}_{\Pi}$ are listed in Fig. 1, where for a context $\Gamma = x_1{:}A_1, ..., x_n{:}A_n$, $FV(\Gamma) = \{x_1, ..., x_n\}$. Some explanations of the rules are in order.

– For linear types $A \multimap B$ and ordered types B/A and $A \setminus B$, there are respective λ-binders $\overline{\lambda}$, λ^r and λ^l, and application operators $\overline{app}$, app^r and

app^l, as summarised in Table 1 in the Introduction (there are no dependent Π-types here.)

- In the three elimination rules, there are side conditions $FV(\Gamma) \cap FV(\Delta) = \emptyset$, saying that the contexts do not share common variables. This is necessary because the calculus is resource-sensitive and the free variables in a well-typed term only occurs once.
- By the lexicon rule, if $(c, A) \in \textsc{Lex}$, constant c has type A.

The calculus has the following conversion rules,[3] where $[a/x]b$ is the term obtained from b by replacing all of the free occurrences of x by a, and $FV(M)$ is the set of free variables that occur in M.

- For linear types:
  - (1) β-conversion: $\overline{app}(\overline{\lambda}x{:}A.b,\ a) \simeq [a/x]b$.
  - (2) η-conversion: $\overline{\lambda}x{:}A.\overline{app}(M, x) \simeq M$, where $x \notin FV(M)$.
- For ordered types:
  - (3) β-conversion: $app^r(\lambda^r x{:}A.b, a) \simeq app^l(a, \lambda^l x{:}A.b) \simeq [a/x]b$.
  - (4) η-conversion: $\lambda^r x{:}A.app^r(M, x) \simeq \lambda^l x{:}A.app^l(x, M) \simeq M$, where $x \notin FV(M)$.

For illustration, let's consider the introduction and elimination rules for $A \setminus B$ in HTLG [14] (see, for example. Figure 1 of [25]), where $\circ$ is the strings' concatenation operator, where Γ and Δ do not share common variables:

$$(\setminus I)\quad \frac{\Gamma, \pi : A \vdash \pi \circ f : B}{\Gamma \vdash f : A \setminus B} \qquad\qquad (\setminus E)\quad \frac{\Delta \vdash a : A \quad \Gamma \vdash f : A \setminus B}{\Delta, \Gamma \vdash a \circ f : B}$$

Informally, for $A \setminus B$, string concatenation corresponds to the application operation app^l. In $\bar{\lambda}_{\rightarrow}$, the following rules are derivable:

$$\frac{\Gamma, x : A \vdash x \circ f : B}{\Gamma \vdash \lambda^l x{:}A.app^l(x, f) : A \setminus B} \qquad \frac{\Delta \vdash a : A \quad \Gamma \vdash f : A \setminus B}{\Delta, \Gamma \vdash app^l(a, f) : B}$$

The second rule corresponds to $(\setminus E)$ rule, while the term in the concludion of the first rule is η-convertible to f and, hence, the rule corresponds to $(\setminus I)$. For instance, we may have a lexicon that contains $(John, \mathrm{NP})$ and $(runs, \mathrm{NP} \setminus \mathrm{S})$. Then, in $\bar{\lambda}_{\rightarrow}$ we have that, from the two judgements in (5), one can deduce the judgement in (6), from which we can deduce that in (7).

(5) $x{:}\mathrm{NP} \vdash x : \mathrm{NP}$ and $\langle\rangle \vdash runs : \mathrm{NP} \setminus \mathrm{S}$

(6) $x{:}\mathrm{NP} \vdash app^l(x, runs) : \mathrm{S}$

(7) $\langle\rangle \vdash \lambda^l x{:}\mathrm{NP}.app^l(x, runs) : \mathrm{NP} \setminus \mathrm{S}$

By η-conversion, $\lambda^l x{:}\mathrm{NP}.app^l(x, runs) \simeq_\eta runs$, i.e., the term in (7) equals $runs$ of type $\mathrm{NP} \setminus \mathrm{S}$.

[3] Usually, a conversion relation is defined by means of the notion of reduction. For example, we have that $\overline{app}(\overline{\lambda}x{:}A.b,\ a)$ one-step reduces to $[a/x]b$. Then, taking the corresponding reduction relation to be the reflexive and transitive closure of one-step reduction, we can define the corresponding conversion relation to be the equivalence relation generated by reduction. We do not get into these technical details in this paper.

	category (type)
John	NP
runs	NP \ S
quickly	(NP \ S) \ (NP \ S)

Table 2 Lexicon for (8).

3 Terms in Syntactic Analysis and Examples

We propose to use *terms* (rather than contexts) to represent NL phrases or sentences.[4] To do this, there is a straightforward mapping from terms to strings, as given by the following definition, where $\circ$ is the concatenation operation of strings, and we write $\mathrm{nf}(e)$ for the normal form of e.

Definition 2 (mapping ϕ) *The (partial) function ϕ that maps terms to strings are defined as follows:*

- *if $(c, A) \in \textsc{Lex}$, then $\phi(c) = c$;*
- *if $\mathrm{nf}(e) = app(e_1, e_2)$, where $app \in \{\overline{app}, app^r, app^l\}$, such that $\phi(e_1)$ and $\phi(e_2)$ are both defined, then $\phi(e) = \phi(e_1) \circ \phi(e_2)$;*
- *otherwise, $\phi(e)$ is undefined.* □

Intuitively, when $\phi(e)$ is defined, the normal form of the expression e consists of only constants in the lexicon and the application operators $\overline{app}$, app^r, and app^l, and the function ϕ removes the application operators and concatenates the resulting strings together. To see whether a term represents a NL sentence/phrase, we can first normalise the term and then apply the mapping ϕ. Here are some examples.

Example 1 Our first example is the following simple sentence (8), to illustrate how to use terms to represent NL phrases/sentences:

(8) John runs quickly.

The lexicon for (8) is given in Table 2. It is straightforward to have:

$$app^l(John, app^l(runs, quickly)) : \mathrm{S}$$

When applying function ϕ to the above term, we have:

$$\phi(app^l(John, app^l(runs, quickly))) = John \circ runs \circ quickly$$

that is, the function ϕ removes app^l and concatenates $\phi(John) = John$, $\phi(runs) = runs$ and $\phi(quickly) = quickly$ together. □

[4] As far as we are aware, this was first considered by Oehrle [29] and related work in categorial analysis, although the notions of term are slightly different. For example, for ordered types, people use λ-strings instead of more standard λ-terms as their objects (this latter idea of employing λ-terms for ordered types was proposed in [18]).

	category (type)
John	NP
saw	(NP \ S)/NP
everyone	NP
yesterday	(NP \ S) \ (NP \ S)

Table 3 Lexicon for (9).

	category (type)
a	NP/CN
very	(CN/CN)/(CN/CN)
book	CN

Table 4 Lexicon for (14).

Example 2 Our second example (9) is taken from Kubota and Levine's book [15] (Section 2.3, p.29):

(9) John saw everyone yesterday.

The lexicon for (9) is given in Table 3. We can derive (10) and (11), from which (12) can be derived, which reduces to (13).

(10) $\overline{\lambda} f : \mathrm{NP} \multimap \mathrm{S}.\ \overline{app}(f, everyone)\ :\ (\mathrm{NP} \multimap \mathrm{S}) \multimap \mathrm{S}$

(11) $\overline{\lambda} \sigma : \mathrm{NP}.\ app^l(John, app^l(app^r(saw, \sigma), yesterday))\ :\ \mathrm{NP} \multimap \mathrm{S}$

(12) $\overline{app}((10), (11))\ :\ \mathrm{S}$

(13) $app^l(John, app^l(app^r(saw, everyone), yesterday))\ :\ \mathrm{S}$

When applying the function ϕ to the above term in (13), we get

$$John \circ saw \circ everyone \circ yesterday$$

the string that corresponds to the sentence (9). □

Example 3 This example (14) is taken from Moot and Retoré's book [24] (Section 2.5, p33), where it is used to illustrate the problem with the empty context. The lexicon for phrase (14) is given in Table 4.

(14) (#) a very book

In the calculi such as the Lambek calculus [16], the introduction rule for types B/A, has a side condition that the relevant context is non-empty. If this side-condition is not abided by, one would be able to derive any logical tautology in the empty context and, in particular, one ould have $\langle\rangle \vdash \mathrm{CN/CN}$. Therefore, one would be able to derive (15) which, if we used contexts (also called the antecedent of a sequent, i.e., the sequence on the left of $\vdash$) to represent NL phrases or sentences, would say that the phrase (14) is a legitimate phrase.

(15) $a : \mathrm{NP/CN},\ very : (\mathrm{CN/CN})/(\mathrm{CN/CN}),\ book : \mathrm{CN}\ \vdash\ \mathrm{NP}$

In our calculus, we'd be able to derive (16) which, compared with (15), has the additional term in (16) that inhabits NP.

(16) $app^r(a, app^r(app^r(very, \lambda^r x{:}\mathrm{CN}.x), book)) \ : \ \mathrm{NP}$

This term is in the normal form (it cannot be further reduced) and the function ϕ is undefined for this term, because $\phi(\lambda^r x{:}\mathrm{CN}.x)$ is undefined. In the terms-as-readings paradigm, this term in (16) does not represent a legitimate phrase; in other words, we cannot incorrectly conclude the legitimacy of (14). □

4 $\bar{\lambda}_{\Pi}$: Substructural Dependent Function Types

Dependent types were first introduced by Martin-Löf in his intuitionistic type theory [21,22]. In this paper, we introduce dependent types in Lambek calculi for syntactic analysis and, in particular, study the system $\bar{\lambda}_{\Pi}$.[5] For example, let CN to be the type of common nouns which themselves may be considered as types themselves. For a variable $X : \mathrm{CN}$, $X \setminus S$ is a type that depends on the object X. (X is an object of type CN as well as a type itself.)

Whence dependent types are introduced, types and objects cannot be syntactically separated anymore, since objects may now occur in type expressions (e.g., X in $X \setminus S$ in the above example). Therefore, one cannot define what types are (cf., Definition 1) independently of well-typedness of terms. As a consequence, one needs to introduce new judgement forms such as $\Gamma \vdash A$ *type* to say that A is a well-formed type, whose derivation is a part of the inference system, because A may contain objects whose well-typedness can only be shown by the existence of a derivation in that system.[6]

The inference rules of $\bar{\lambda}_{\Pi}$ are listed in Fig. 2 where, as in $\bar{\lambda}_{\rightarrow}$, the lexicon Lex is a finite set of pairs (c, A), but now it satisfies the property that, if $(c, A) \in \textsc{Lex}$, then A is a type, i.e., $\langle\rangle \vdash A$ *type* is derivable in $\bar{\lambda}_{\Pi}$.

We now give some explanations of $\bar{\lambda}_{\Pi}$, including its rules in Fig. 2. First, substructural Π-types subsume those substructural function types in $\bar{\lambda}_{\rightarrow}$ described in Section 2. When the variable x does not occur free in B, the dependent substructural types $\overline{\Pi} x{:}A.B$, $\Pi^r x{:}A.B$ and $\Pi^l x{:}A.B$ degenerate to their corresponding non-dependent versions $A \multimap B$, A/B, and $A \setminus B$, respectively. Put in another way, the non-dependent substructural function types are just special cases of their respective dependent substructural Π-types.

In a dependent type theory, one may introduce *universes*, which are types of types. Put in another way, every object in a universe is itself a type. In $\bar{\lambda}_{\Pi}$,

[5] This work is based on the first author's previous work [18,20], but $\bar{\lambda}_{\Pi}$ does not contain intuitionistic types which are considered in [18,20] at the same time. Also, $\bar{\lambda}_{\Pi}$ is different technically in that: (1) in [18], types cannot depend on substructural variables, while in $\bar{\lambda}_{\Pi}$ that is allowed; (2) in [20], only linear types are considered, while in $\bar{\lambda}_{\Pi}$ ordered typed are studied as well.

[6] The notion of derivation is a basic one in formal systems and their meta-theories. For type theory, it is defined as: a derivation of a judgement J is a finite sequence $J_1, ..., J_k$ of judgements ($k \geq 1$) such that $J_k = J$ and, for any $1 \leq i \leq k$, there exists an instance of an inference rule whose conclusion is J_i and whose premises are all in the set $\{ J_j \mid j < i \}$.

Contexts

$$\frac{}{\langle\rangle\ \mathit{valid}} \qquad \frac{\Gamma \vdash A\ \mathit{type} \quad x \notin FV(A)}{\Gamma,\ x{:}A\ \mathit{valid}}$$

Variables

$$(Var) \qquad \frac{\Gamma, x{:}A\ \mathit{valid}}{\Gamma, x{:}A \vdash x{:}A} \quad (\forall y \in FV(\Gamma).\ x \sim_{\Gamma, x:A} y)$$

where, in the side condition of (Var)*, for any* $\Delta = x_1{:}A_1, ..., x_n{:}A_n$*, the dependency relation* $\sim_\Delta$ *is defined as:*
(1) if $y \in FV(A_i)$*, then* $x_i \sim_\Delta y$*; (2) if* $x \sim_\Delta y$ *and* $y \sim_\Delta z$*, then* $x \sim_\Delta z$*.*

Formation rules for substructural Π*-types*

$$\frac{\Gamma,\ x{:}A \vdash B\ \mathit{type}}{\Gamma \vdash \Pi^r x{:}A.B\ \mathit{type}} \qquad \frac{\Gamma,\ x{:}A \vdash B\ \mathit{type}}{\Gamma \vdash \Pi^l x{:}A.B\ \mathit{type}} \qquad \frac{\Gamma,\ x{:}A \vdash B\ \mathit{type}}{\Gamma \vdash \overline{\Pi} x{:}A.B\ \mathit{type}}$$

Ordered Π*-types* ((app^r) and (app^l) have the side condition $FV(\Gamma) \cap FV(\Delta) = \emptyset$.)

$$(\lambda^r) \quad \frac{\Gamma,\ x{:}A \vdash b : B}{\Gamma \vdash \lambda^r x{:}A.b : \Pi^r x{:}A.B} \qquad (app^r) \quad \frac{\Gamma \vdash f : \Pi^r x{:}A.B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash app^r(f, a) : [a/x]B}$$

$$(\lambda^l) \quad \frac{\Gamma,\ x{:}A \vdash b : B}{\Gamma \vdash \lambda^l x{:}A.b : \Pi^l x{:}A.B} \qquad (app^l) \quad \frac{\Gamma \vdash f : \Pi^l x{:}A.B \quad \Delta \vdash a : A}{\Delta, \Gamma \vdash app^l(a, f) : [a/x]B}$$

Linear Π*-types* ($(\overline{app})$ has the side condition $FV(\Gamma) \cap FV(\Delta) = \emptyset$.)

$$(\bar{\lambda}) \quad \frac{\Gamma,\ x{:}A \vdash b : B}{\Gamma \vdash \bar{\lambda} x{:}A.b : \overline{\Pi} x{:}A.B} \qquad (\overline{app}) \quad \frac{\Gamma \vdash f : \overline{\Pi} x{:}A.B \quad \Delta \vdash a : A}{\Gamma, \Delta \vdash \overline{app}(f, a) : [a/x]B}$$

The lexicon rule

$$\frac{(c, A) \in \textsc{Lex}}{\langle\rangle \vdash c : A}$$

*Basic Categories (*S *and* CN *being universes)*

$$\frac{}{\langle\rangle \vdash \mathrm{S}\ \mathit{type}} \qquad \frac{\Gamma \vdash P : \mathrm{S}}{\Gamma \vdash P\ \mathit{type}} \qquad \frac{}{\langle\rangle \vdash \mathrm{NP}\ \mathit{type}} \qquad \frac{}{\langle\rangle \vdash \mathrm{CN}\ \mathit{type}} \qquad \frac{\Gamma \vdash A : \mathrm{CN}}{\Gamma \vdash A\ \mathit{type}}$$

The conversion/cumulativity rule

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash B\ \mathit{type}}{\Gamma \vdash a : B} \quad (A \preceq B)$$

Fig. 2 Inference rules of $\bar{\lambda}_\Pi$

we have two universes: S and CN. For example, intuitively, an object in CN is a type that corresponds to a common noun. The corresponding rules stipulate that, for example, if A is of type CN, we can derive A *type*.

Unlike the calculus with only non-dependent types (see Figure 1 in Section 2), in the (Var) rule in Figure 2, the context Γ may be non-empty because of type dependency. For instance, for A *type* and $x{:}A \vdash B(x)$ *type*, we can de-

rive (17); taking A and $B(x)$ to be CN and X, respectively, we will have (18):

(17) $x{:}A, y{:}B(x) \vdash y : B(x)$

(18) $X{:}\mathrm{CN}, y{:}X \vdash y : X$

Here, we say that x in (17) (and X in (18)) *essentially* occurs free in y: it does not literally occur free in y, but it occurs free in y's contextual type. (See Theorem 5 in Section 5.)

$\bar{\lambda}_{\Pi}$ has the following conversion rules (they are the 'same' as those for $\bar{\lambda}_{\rightarrow}$, but note that we now have dependent types – we repeat them here for completeness):

– For linear Π-types:

(19) β-conversion: $\overline{app}(\overline{\lambda}x{:}A.b,\ a) \simeq [a/x]b$.

(20) η-conversion: $\overline{\lambda}x{:}A.\overline{app}(M, x) \simeq M$, where $x \notin FV(M)$.

– For ordered Π-types:

(21) β-conversion: $app^r(\lambda^r x{:}A.b, a) \simeq app^l(a, \lambda^l x{:}A.b) \simeq [a/x]b$.

(22) η-conversion: $\lambda^r x{:}A.app^r(M, x) \simeq \lambda^l x{:}A.app^l(x, M) \simeq M$, where $x \notin FV(M)$.

The cumulativity rule (the last rule in Figure 2) employs the cumulativity relation $\preceq$, which defines the notion of subtyping between types. It is based on the conversion relation as specified above and induced by a lexicon Lex such that, if $(c, \mathrm{CN}) \in$ Lex, then $c \preceq \mathrm{NP}$.

Definition 3 (cumulativity relation) *The cumulativity relation $\preceq$ is defined to be the smallest binary relation over terms such that*

1. *$\preceq$ is a partial order w.r.t. conversion, that is,*
   (a) *if $A \simeq B$, then $A \preceq B$;*
   (b) *if $A \preceq B$ and $B \preceq A$, then $A \simeq B$; and*
   (c) *if $A \preceq B$ and $B \preceq C$, then $A \preceq C$.*
2. *If $(c, \mathrm{CN}) \in$ Lex, then $c \preceq \mathrm{NP}$.*
3. *If $A' \preceq A$ and $B \preceq B'$, then $\Pi^\circ x{:}A.B \preceq \Pi^\circ x{:}A'.B'$, where $\Pi^\circ \in \{\Pi^r, \Pi^l, \overline{\Pi}\}$.*

As special cases of the third clause above, we have:[7]

– *If $A' \preceq A$ and $B \preceq B'$, then $A \longrightarrow B \preceq A' \longrightarrow B'$, where $\longrightarrow \in \{\multimap, /, \backslash\}$.*

□

[7] Note that the above $\longrightarrow$ is one of the substructural arrows, not the intuitionistic arrow. See Footnote 1.

	category (type)
most	$\Pi^r X{:}\mathrm{CN}.\ \mathrm{S}/(X \setminus \mathrm{S})$
students	CN
study	$\mathrm{NP} \setminus \mathrm{S}$
hard	$(\mathrm{NP} \setminus \mathrm{S}) \setminus (\mathrm{NP} \setminus \mathrm{S})$

Table 5 Lexicon for (23).

Note that, the cumulativity rule subsumes the following conversion rule:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash B\ type}{\Gamma \vdash a : B} \quad (A \simeq B)$$

The above rule is essential for all dependent type theories, even without subtyping. It says that, if two terms are convertible, then two types which are only different because of these two terms are the same. Meta-theoretic properties of the calculus $\bar{\lambda}_\Pi$ are briefly discussed Section 5.

Here is an example that uses dependent substructural types in $\bar{\lambda}_\Pi$ for syntactical analysis.

Example 4 The following example (23), taken from [19], illustrates how the dependent types in $\bar{\lambda}_\Pi$ may be used. The lexicon for (23) is given in Table 5.

(23) Most students study hard.

According to Table 5 and the inference rules in $\bar{\lambda}_\Pi$ (Figure 2), we can derive the following judgement:

(24) $app^r(app^r(most, students), app^l(study, hard)) : \mathrm{S}$

Note that the above judgement (24) is derivable partly because $students \leq \mathrm{NP}$ and, therefore, $\mathrm{NP} \setminus \mathrm{S} \leq students \setminus \mathrm{S}$.

Applying ϕ to the term in (24), we get

$$\begin{aligned} &\phi(app^r(app^r(most, students), app^l(study, hard))) \\ &= \mathrm{most} \circ students \circ study \circ hard \end{aligned}$$

In other words, (23) is a sentence. □

5 Remarks on Meta-Theoretic Properties

In this paper, we have not detailed the technical machinery for meta-theoretic studies (see, for example, Footnote 3). Here, we only give some brief remarks about the meta-theoretic properties of $\bar{\lambda}_\rightarrow$ and $\bar{\lambda}_\Pi$.

First, partly because that the formulation of the system follows the more standard ways, the expected properties hold. For example, if we write the notion of reduction as $M \rhd N$, we have the following property of subject reduction (or preservation of typing):

– If $\Gamma \vdash a : A$ and $a \rhd b$, then $\Gamma \vdash b : A$.

It is worth remarking that subject reduction is not only important in meta-theoretic studies, but also very useful in practical applications (in fact, we have made implicit use of this when explaining the calculi in examples.)

The systems $\bar{\lambda}_{\rightarrow}$ and $\bar{\lambda}_{\Pi}$ are substructural in the sense that, when a term is well-typed in a context, the free occurrences of the contextual variables in the term are resource-sensitive, as described in the theorem below. In $\bar{\lambda}_{\rightarrow}$ (without dependent types), as expected, every contextual variable occurs free for exactly once in a well-typed term. In $\bar{\lambda}_{\Pi}$, however, things are more complex, because of the presence of dependent types. As a consequence, we show that the *essential* free occurrences abide by the linearity property (see the remarks in the paragraph about examples (17-18) in Section 4.).

Before describing it, we need to define a notion of *essential occurrence* to be used.

Definition 4 (essential occurrence) *If $\Gamma \vdash a : A$ in $\bar{\lambda}_{\Pi}$, the multiset $E_{\Gamma}(a)$ of variables that occur free essentially in the term a is defined by induction on derivations as follows:*

1. *For the inference rule (Var),*

$$E_{\Gamma,x:A}(x) = FV(A) \cup \{x\}.$$

2. *For the λ-abstraction rules (λ^r), (λ^l) and $(\overline{\lambda})$,*

$$E_{\Gamma}(\lambda x{:}A.b) = E_{\Gamma,x:A}(b) \backslash \{x\},$$

   where $\lambda \in \{\overline{\lambda}, \lambda^r, \lambda^l\}$.
3. *For the application rules (app^r), (app^l) and $(\overline{app})$,*

$$E_{\Gamma_1,\Gamma_2}(app(M,N)) = E_{\Gamma_1}(M) \cup E_{\Gamma_2}(N),$$

   where $app \in \{\overline{app}, app^r, app^l\}$. □

Now, we can state the linearity theorem.

Theorem 5 (linearity)

- *(Lineariry in $\bar{\lambda}_{\rightarrow}$) In $\bar{\lambda}_{\rightarrow}$, every contextual variable occurs free for exactly once in a well-typed term. In symbols, if $\Gamma \vdash a : A$ with $\Gamma = x_1{:}A_1, ..., x_n{:}A_n$, then each x_i occurs free in a for exactly once $(i = 1, ..., n)$.*

- *(Weak linearity in $\bar{\lambda}_{\Pi}$) In $\bar{\lambda}_{\Pi}$, every contextual variable occurs free essentially for exactly once in a well-typed term. In symbols, if $\Gamma \vdash a : A$ with $\Gamma = x_1{:}A_1, ..., x_n{:}A_n$, then each x_i occurs free essentially in a for exactly once (i.e., $x_i \in E_{\Gamma}(a)$ for exactly once $(i = 1, ..., n)$).*

Proof By induction on derivations. □

Remark 1 The weak linearity property in the above theorem is similar to that for the linear type system LDTT [20] where we also consider intuitionistic types, whose presence has made the formulation of LDTT much more complicated than that of $\bar{\lambda}_{\Pi}$ in this paper (cf., Footnote 1). □

6 Conclusion

In this paper, we have studied a substructural type theory with both linear and ordered types and their corresponding dependent types and showed its applications in categorial syntactical analysis.

One may consider further extensions of the calculus, an example of which is substructural ordered Σ-types, as first considered in [18], which are objects of the universe CN, whose formation is governed by the following rules:

$$\frac{A : \mathrm{CN} \quad \vdash B : A \backslash \mathrm{S}}{\Sigma^{\sim}(A, B) : \mathrm{CN}} \qquad \frac{A : \mathrm{CN} \quad B : \mathrm{S}/A}{\Sigma^{\circ}(A, B) : \mathrm{CN}}$$

For example, the phrase (25) concerns with adjectival modification 'diligent students'.

(25) All diligent students got good marks.

In a relevant lexicon, *diligent* is of type $\mathit{human} \backslash \mathrm{S}$, where $\mathit{human} : \mathrm{CN}$, and *student* is of type CN. The phrase 'diligent students' in (25) can be analysed as $\Sigma^{\sim}(\mathit{student}, \mathit{diligent})$, which is of type CN (note that it is legitimate because *student* is a subtype of *human*) and the mapping from terms to strings can be extended accordingly to incorporate ordered Σ-types [18,7]. However, it should be pointed out that, in [18], types cannot depend on substructural variables. To do so may have unexpected implications when introducing other dependent types than Π-types (as we have done in this paper). Future work is called for to work out the details of such an extension.

As some have argued (see, for example, [12]), one of the attractive features of categorial grammar is the related nice account of syntax/semantics interface. Studying substructural dependent types opens up a possibility of considering the syntactical counterpart of formal semantics based on dependent type theories, including MTT-semantics [17,7] and Dependent Type Semantics [3,4]. New studies are also required to look into this for future development.

References

1. Barker, C.: NL_{λ} as the logic of scope and movement. J of Logic, Language and Information 28(2) (2019)
2. Barker, C., Shan, C.C.: Continuations and Natural Language. Oxford University Press (2015)
3. Bekki, D.: Representing anaphora with dependent types. In: International conference on logical aspects of computational linguistics. pp. 14–29. Springer (2014)
4. Bekki, D., Mineshima, K.: Context-Passing and Underspecification in Dependent Type Semantics. In: [6] (2017)

5. Cervesato, I., Pfenning, F.: A linear logical framework. Information and Computation 179 (2002)
6. Chatzikyriakidis, S., Luo, Z. (eds.): Modern Perspectives in Type-Theoretical Semantics. Springer (2017)
7. Chatzikyriakidis, S., Luo, Z.: Formal Semantics in Modern Type Theories. Wiley/ISTE (2020)
8. Curry, H., Feys, R.: Combinatory Logic, vol. 1. North Holland Publishing Company (1958)
9. Girard, J.Y.: Linear logic. Theoret. Comput. Sci. 50 (1987)
10. de Groote, P.: Towards abstract categorial grammars. Proc. of the 39th Annual Meeting of the Association for Computational Linguistics (2001)
11. Howard, W.A.: The formulae-as-types notion of construction. In: Hindley, J., Seldin, J. (eds.) To H. B. Curry: Essays on Combinatory Logic. Academic Press (1980)
12. Jacobson, P.: The syntax/semantics interface in categorial grammar. In: Lappin, S. (ed.) The Handbook of Contemporary Semantic Theory (1996)
13. Krishnaswami, N., Pradic, P., Benton, N.: Integrating dependent and linear types. POPL 2015
14. Kubota, Y., Levine, R.: Gapping as like-category coordination. Logical Aspects of Computational Linguistics 2012 (LACL12). LNCS 7351 (2012)
15. Kubota, Y., Levine, R.: Type-Logical Syntax. MIT (2020)
16. Lambek, J.: The mathematics of sentence structure. The American Mathematical Monthly 65(3) (1958)
17. Luo, Z.: Formal semantics in modern type theories with coercive subtyping. Linguistics and Philosophy 35(6), 491–513 (2012)
18. Luo, Z.: A Lambek Calculus with Dependent Types. Extended Abstract in Types for Proofs and Programs (TYPES 2015) Tallinn (2015)
19. Luo, Z.: Modern Type Theories: Their Development and Applications. Tsinghua University Press (2022), (In press)
20. Luo, Z., Zhang, Y.: A linear dependent type theory. Extended Abstract in Types for Proofs and Programs (TYPES 2016) Novi Sad (2016)
21. Martin-Löf, P.: An intuitionistic theory of types: predicative part. In: H.Rose, J.C.Shepherdson (eds.) Logic Colloquium'73 (1975)
22. Martin-Löf, P.: Intuitionistic Type Theory. Bibliopolis (1984)
23. Moortgat, M.: Typelogical grammar. Stanford Encyclopedia of Philosophy (2010)
24. Moot, R., Retoré, C.: The Logic of Categorial Grammar. Springer, LNCS 6850 (2012)
25. Moot, R., Stevens-Guille, S.: Proof-theoretic aspects of hybrid type-logical grammars. Proc of the 24th Inter. Conf. on Formal Grammar (FG 2019), LNCS 11668 (2019)
26. Morrill, G.: Categorial Grammar: Logical Syntax, Semantics, and Processing. Oxford University Press (2011)
27. Morrill, G., Valentín, O., Fadda, M.: The displacement calculus. J of Logic, Language and Information 20(1) (2011)
28. Muskens, R.: Categorial grammar and lexical-functional grammar. Proc. of the LFG01 Conference (2001)
29. Oehrle, R.: Term-labeled categorial type systems. Linguistics and Philosophy 17(6) (1994)
30. Polakow, J., Pfenning, F.: Natural deduction for intuitionistic non-commutative linear logic. Proc. of Typed Lambda Calculus and Applications (TLCA 1999), LNCS 1581 (1999)
31. Vákár, M.: A categorical semantics for linear logical frameworks. FoSSaCS 2015